

IntraComms

LAN-Based Internal Communication System

Project Report: Final Year Project - Semester 3
Author: Ibrahim Habib
Student ID: LC000111000882
Institution: Lincoln University College
Program: Computer Software Engineering
Date: June 2026

Flask

Socket.IO

AES-256-GCM

SQLite

ECDH P-256

Abstract

IntraComms is a self-hosted, offline-first communication platform that lets every member of a local area network (LAN) exchange real-time text messages, share files of any size, send voice messages, and place voice calls - all without any dependency on the public internet or any third-party cloud service. The system is built on Python, Flask, and Flask-SocketIO on the server, with a dependency-free vanilla-JavaScript client that implements the Engine.IO v4 WebSocket framing protocol by hand and performs all cryptography in the browser using the Web Crypto API.

The defining characteristic of IntraComms is that confidentiality is enforced end-to-end: direct messages and direct file transfers are encrypted in the sender's browser and only ever decrypted in the recipient's browser. The server stores ciphertext and never has access to plaintext, message keys, or even the original file names. Identity is established with per-user elliptic-curve (ECDH P-256) key pairs whose private halves never leave the device, and users can verify each other through a short out-of-band fingerprint to defeat man-in-the-middle attacks.

This report documents the complete project: its motivation and objectives, the requirements it satisfies, its system design and architecture, the security model, the implementation of each feature, the testing performed, and an honest account of the engineering challenges encountered - most notably the redesign of the large-file transfer subsystem from a WebRTC peer-to-peer data channel (which stalled and ran slowly across mixed wired/wireless segments) to a server-relayed, end-to-end-encrypted "FastTransfer" protocol that streams large files to disk and imposes effectively no size limit.

1. Introduction

1.1 Background

Modern team-communication tools such as Slack, Microsoft Teams, and WhatsApp are excellent, but they share two assumptions that do not always hold: they require a working internet connection, and they route private data through servers owned by a

third party. There are many environments where one or both of these assumptions is unacceptable - a factory floor or hospital wing with no outbound internet, a computer-laboratory or examination hall that must remain air-gapped, a field site, a ship, or simply an organisation with a strict data-locality policy that forbids internal communication from leaving the building.

IntraComms targets exactly this gap: a communication system that runs entirely on one machine inside a LAN, that any browser on that LAN can reach, and that keeps every byte of user data within the network perimeter.

1.2 Problem Statement

Build a secure, real-time, multi-user communication system that:

- operates with zero internet connectivity, on commodity hardware;
- requires no client installation beyond a modern web browser;
- protects message and file confidentiality even from the server operator;
- handles not just text but images, video, audio, documents, and large files; and
- remains simple enough to be understood, deployed, and maintained by a single developer.

1.3 Aim and Objectives

The aim of the project is to design and implement a production-quality, LAN-only, end-to-end-encrypted communication platform. The concrete objectives, and their final status, are listed below.

#	Objective	Status
1	Real-time messaging over a LAN	Complete
2	Direct (one-to-one) and broadcast	Complete
3	(optional) distributed login, and session	Complete
4	Role-based access control (admin / user)	Complete
5	File upload, storage, and download	Complete
6	Adaptive media pipeline for images, video, and audio	Complete
7	End-to-end encryption for direct	

messages (ECIES + AES-256-GCM)

Complete

#	Objective	Status
8	End-to-end encrypted direct file transfer	Complete
9	Large-file transfer with effectively no latency	Complete
10	Public-key fingerprint verification	Complete
11	(anti-MIM) messages (recorded, replayed)	Complete
12	Real-time voice calls	Complete
13	Desktop notifications for incoming messages	Complete
14	HTTPS / WSS transport with an optional TLS certificate	Complete
15	Pre-generated TLS certificate and soft-delete	Complete

1.4 Scope

IntraComms is a single-server application intended for a trusted LAN of up to a few dozen concurrent users. It deliberately does not attempt horizontal scaling, federation between sites, or operation over the open internet. Group (many-to-many) end-to-end encryption is out of scope; broadcast messages are therefore plaintext by design, since they are public to the whole network.

1.5 Report Organisation

Section 2 reviews the relevant background and the technologies chosen. Section 3 states the functional and non-functional requirements. Section 4 presents the system design, and Section 5 the detailed system architecture. Section 6 is devoted to the security architecture. Section 7 describes the implementation of each feature, and Section 8 the implementation details and notable engineering decisions. Section 9 covers testing and validation, Section 10 discusses results, and Section 11 the engineering challenges and their solutions. Sections 12 to 14 cover limitations, future work, and the conclusion. Appendices provide a full API reference, the Socket.IO event catalogue, the database schema, and a deployment guide.

2. Background and Technology Review

2.1 The LAN Communication Context

A LAN-only design changes the engineering trade-offs compared to an internet-scale service. Latency between peers is sub-millisecond and packet loss is near zero, so protocols tuned for the lossy public internet (for example the SCTP congestion control underneath WebRTC data channels) can actually perform worse than plain TCP. Bandwidth is high but shared, and the server is the one host that every client is guaranteed to be able to reach. These observations directly shaped the final large-file transfer design (Section 7.4).

2.2 Comparison with Existing Solutions

Property	Slack / Teams	Matrix (self-host)	IntraComms
Works fully offline (LAN)	No	Partially	Yes
No third-party servers	No	Yes	Yes
End-to-end encryption	Limited	Optional	Default for DMs
Zero client install	No (app)	No (app)	Yes (browser)
Single-binary deployment	No	No	Yes (one process)

IntraComms does not aim to match the feature breadth of these products; it aims to occupy the niche - offline, self-contained, private-by-default - that none of them serves well.

2.3 Cryptographic Primitives

The security design rests on three standard, well-analysed primitives, all provided by the browser's Web Crypto API and the Python cryptography library:

- ECDH on curve P-256 (NIST secp256r1) for key agreement. Each user has a long-term key pair; the private key never leaves the browser.
- AES-256-GCM for authenticated symmetric encryption. GCM provides both confidentiality and integrity: any tampering with the ciphertext makes decryption fail.
- ECIES (Elliptic Curve Integrated Encryption Scheme) as the composition used to "seal" a symmetric key to a recipient's public key: generate an ephemeral ECDH key pair, derive a shared AES key from the ephemeral private key and the recipient's

public key, encrypt the payload, and transmit the ephemeral public key alongside the ciphertext.

Passwords are protected separately with PBKDF2-HMAC-SHA256 via Werkzeug.

2.4 Real-Time Web Transport

Real-time delivery uses WebSockets. Rather than depend on the bundled socket.io.js client library, the IntraComms browser client implements the Engine.IO v4 (EIO4) framing protocol directly: it performs the handshake, answers ping/pong heartbeats, and encodes and decodes the 42[event, payload] event frames by hand. This removes a third-party dependency and demonstrates a clear understanding of the underlying protocol. On the server, Flask-SocketIO (backed by the eventlet asynchronous worker) provides the event dispatch and the room abstraction used for message routing.

2.5 Technology Selection Rationale

Concern	Choice	Why
Web framework	Flask 3.1	Minimal, well documented, ideal for a
Real-time	Flask-SocketIO	Mature WebSocket layer with rooms
Async worker	eventlet	and fallbacks to async mode for
Persistence	SQLite	Zero-config, single-file, perfect for a
ORM	SQLAlchemy	LAN node Parameterised queries
Auth	Flask-Login	Battle-tested session management
Crypto (server)	cryptography 44	X.509 cert generation, key handling
Crypto (client)	Web Crypto API	Native, audited, no JS crypto library
Media	Pillow + FFmpeg	to trust re-encode and video/audio
Frontend	Vanilla JS + CSS	No framework; small, fast, fully
		understood

3. Requirements Analysis

3.1 Functional Requirements

- FR-1 Users can register with a username, email, and password; the first registered user becomes the administrator.
- FR-2 Users can log in and log out; sessions persist via a signed cookie.
- FR-3 Users can send direct messages to a single recipient and broadcast messages to all users.
- FR-4 Direct messages are end-to-end encrypted; broadcast messages are plaintext.
- FR-5 Users can share files - images, video, audio, documents, archives.
- FR-6 Direct file transfers are end-to-end encrypted.
- FR-7 The system supports large file transfers with no practical size limit.
- FR-8 Users can record and send encrypted voice messages.
- FR-9 Users can place and receive real-time voice calls.
- FR-10 Users can verify each other's identity via a public-key fingerprint.
- FR-11 Users receive desktop notifications for incoming messages, calls, and file offers when the application window is not focused.
- FR-12 Administrators can activate, deactivate, and change the role of any user.
- FR-13 Senders can delete their own messages (soft delete).
- FR-14 The UI shows who is currently online.

3.2 Non-Functional Requirements

- NFR-1 (Security) The server must never have access to plaintext direct messages, direct file contents, file keys, or original file names.
- NFR-2 (Confidentiality in transit) All traffic uses HTTPS/WSS.
- NFR-3 (Availability) Per-user rate limiting and automatic reconnection with exponential backoff protect the service from overload and transient drops.
- NFR-4 (Usability) The client must run in any modern browser with no install and must be responsive on both desktop and mobile.
- NFR-5 (Portability) The entire system must run from a single Python process on Windows, Linux, or macOS.
- NFR-6 (Maintainability) The codebase must be small and readable enough for one developer to maintain.

3.3 User Roles and Use Cases

There are two roles. A user can do everything in FR-1 to FR-11 and FR-13 to

FR-14. An administrator additionally has FR-12. Representative use cases:

- Send a private file: User A selects a file in a direct conversation with User B; the client encrypts it and transfers it so that only B can read it.
 - Verify a contact: A and B compare the short fingerprint shown for each other's key over a trusted channel (in person) and mark each other verified.
 - Administer the roster: The admin deactivates a departed user's account so they can no longer log in, without deleting their message history.
-

4. System Design

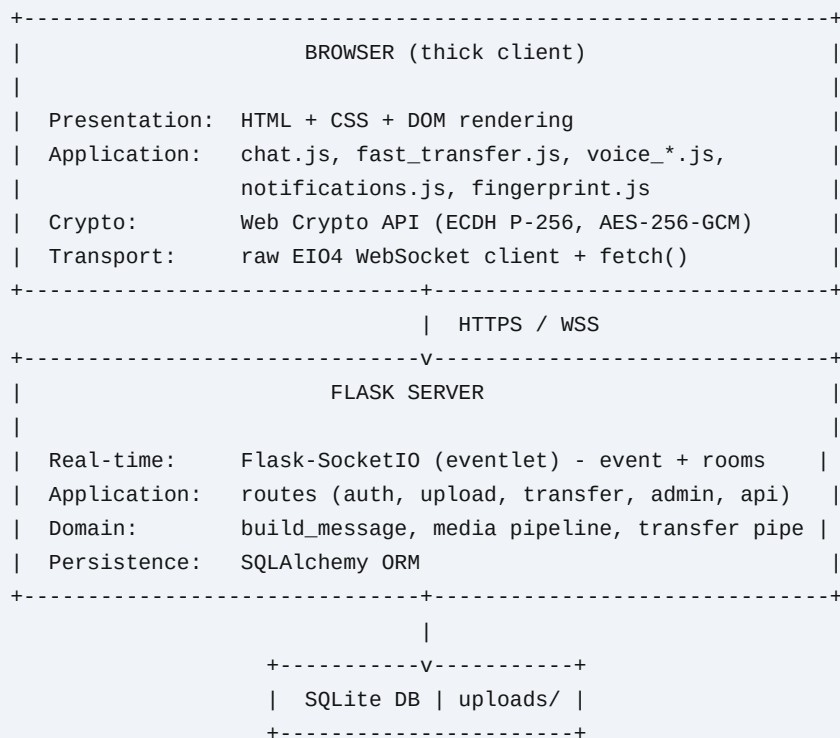
4.1 Design Principles

- Client-heavy, server-blind. All cryptography happens in the browser. The server is a relay and a store of ciphertext; it is deliberately kept ignorant of secrets so that compromising it does not compromise message confidentiality.
- One process, one file of truth. The whole backend lives in server/server.py with the data models in server/models/. This avoids premature abstraction and keeps the system comprehensible.
- Progressive enhancement. Messaging works over a plain HTTP form post if WebSockets are unavailable; encryption is applied when the Web Crypto API and the recipient's key are available.
- Adapt to the network. The client measures its link quality and the server tailors media processing accordingly.

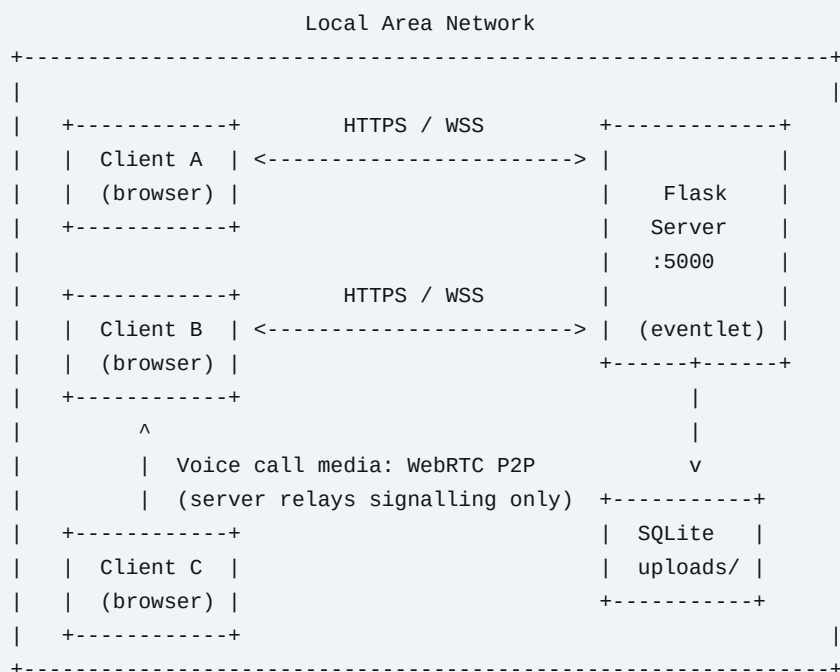
4.2 Architectural Style

The system is a layered, client-server architecture with a thick client.

The browser hosts the presentation layer and the entire cryptographic layer; the Flask server hosts the application/routing layer, the real-time event layer, and the persistence layer.



4.3 High-Level Architecture



Note the single exception to the "everything through the server" rule: live voice-call audio travels peer-to-peer over WebRTC, and the server relays only the

signalling. File transfers, by contrast, are relayed through the server as encrypted shards (see Section 7.4 for the rationale).

4.4 Component Decomposition

Component	Location	Responsibility
App factory	server.py: create_app	Build Flask app, wire extensions
Routes	server.py: register_routes	All HTTP endpoints
Socket handlers	server.py (module level)	All real-time events
Media pipeline	server.py	Image/video/audio processing
Transfer pipe	server.py	FastTransfer shard store-and-forward
Data models	server/models/	User, Message, FileRecord ORM
Chat client	static/js/chat.js	UI, messaging, E2EE orchestration
FastTransfer client	static/js/fast_transfer.js	Large-file shard transfer
Voice call client	static/js/voice_chat.js	WebRTC voice calls
Voice message client	static/js/voice_message.js	Record/play encrypted voice notes
Fingerprint client	static/js/fingerprint.js	Key fingerprint verification UI
Notifications	static/js/notifications.js	Desktop notifications

4.5 Database Design

The schema has three tables: users, messages, and file_records. A user sends and receives many messages and uploads many files; a message may reference one file record.

4.5.1 Schema Migration Strategy

4.6 User-Interface Design

The dashboard is a two-panel layout: a conversation rail on the left (with "Chats" and "Contacts" tabs) and a chat panel on the right. Conversation switching is entirely client-side - all relevant messages are loaded once and the JavaScript shows or hides them by filtering data attributes, so changing chats is instantaneous. All visual tokens (colours, spacing, the 320 px rail width) are CSS custom properties in :root, and below 680 px the layout collapses to a single stacked column with mobile show/hide. The palette is a dark-navy navigation bar over a light message area, with white incoming bubbles and pale-blue outgoing bubbles.

5. System Architecture

5.1 Server Architecture

The server is created by an application factory, `create_app()`, which builds the Flask app, configures it, initialises the database, login manager, and Socket.IO, registers a context processor for cache-busting (Section 8.3), and calls `register_routes()` and `register_cli()`. The real-time event handlers are declared at module level with `@socketio.on(...)` and authenticate using `current_user.is_authenticated`.

The full HTTP surface is summarised below (see Appendix A for details).

Area	Endpoints
Pages	/, /register, /login, /logout, /dashboard, /admin
Messaging	/messages/<id>/delete
Uploads	/upload, /upload/video/chunk, /upload/e2ee
Downloads	/uploads/<id>, /uploads/<id>/thumb, /uploads/<id>/blob,
Transfer	/transfer/<token>/shard/<n> (GET and POST) .../meta
Keys	/api/me/pubkey, /api/me/fingerprint, /api/me/key-backup,
Admin	/admin/users/<id>/activate /api/users/<id>/pubkey
Utility	/ping

5.2 The Real-Time Layer

5.2.1 EIO4 Framing

The browser client speaks the Engine.IO v4 protocol over a raw WebSocket. The handshake and heartbeat are handled by recognising a small set of frame prefixes:

Frame	Meaning	Client response
0{...}	Engine.IO open	Send 40 (Socket.IO connect)
40	Socket.IO connected	Mark connected, flush queued
2	Ping (heartbeat)	Send 3 (pong) messages
42{...}	Socket.IO event	Parse JSON array, dispatch by type

Outgoing events are encoded as 42 + JSON.stringify([eventName, payload]).

5.2.2 Rooms and Routing

On connect, each user joins two rooms: a personal room `user_{id}` (which all of that user's tabs and devices share) and the shared broadcast room. Direct messages and targeted events are emitted to the sender's and recipient's personal rooms; broadcast messages and presence changes are emitted to the broadcast room. Almost all server-to-client traffic is delivered inside a single message event carrying a typed envelope of the form { "type": <string>, "payload": <object> }, which the client's `dispatch()` function routes to the correct handler.

5.3 Client Module Architecture

The client is a set of cooperating modules, each an Immediately-Invoked Function Expression exposing a small public interface. There is no front-end framework and no `socket.io.js`.

Module	Responsibility
E2EE	ECDH key lifecycle; ECIES encrypt/decrypt of messages
FileE2EE	Client-side file encryption and decrypt-on-download
FastTransfer	Large-file sharded transfer (sender and receiver)
VoiceChat	WebRTC voice-call setup and signalling
VoiceMessage	Recording, encrypting, and playing voice notes
Fingerprint	Computing and verifying public-key fingerprints
Notify	Desktop notifications and the navbar mute toggle

Module	Responsibility
Socket layer	EIO4 framing, heartbeat, reconnect with backoff, dispatch

5.4 Request and Data Flow

5.4.1 Sending an Encrypted Direct Message

```

Sender browser:
  E2EE.encryptFor(recipient_pub, plaintext)
    -> ECIES (fresh ephemeral ECDH + AES-256-GCM)
    -> { ciphertext, ephemeral_pub, iv }
  E2EE.encryptFor(own_pub, plaintext)      (sender copy)
  WebSocket: 42["message", { type:"send_message", payload:{...ciphertext...} }]

Server:
  authenticate, rate-limit, build_message(), persist ciphertext,
  emit "message"/new_message to user_{sender} and user_{receiver}

Recipient browser:
  dispatch(new_message) -> render bubble (pending)
  E2EE.decrypt(ephemeral_pub, iv, ciphertext)
    -> ECDH derive AES key -> AES-GCM decrypt -> plaintext into DOM

```

The server stores and forwards opaque ciphertext; it cannot read the message. The separate "sender copy" (the same plaintext encrypted to the sender's own public key) lets senders read back their own history without the recipient's private key.

5.4.2 Presence

On the first WebSocket connection for a user (a per-user connection counter rising from 0 to 1) the server broadcasts `user_online`; when the last connection closes (counter back to 0) it broadcasts `user_offline`. A connecting client is also sent an `online_users` snapshot so it immediately knows who is present.

5.5 Connection Quality and Adaptive Media

The client periodically probes latency with `GET /ping` and, combined with the browser's Network Information API, classifies its link into a tier. The server then selects media-processing parameters from that tier.

Tier	Condition (approx.)	Photo max	Video chunk	Audio bitrate
------	---------------------	-----------	-------------	---------------

Tier	Condition (approx.)	Photo max	Video chunk	Audio bitrate
wired	RTT < 5 ms, > 50	2560 px	20 MB	64 kbps
wifi_good	RTT < 20 ms, > 20	1920 px	5 MB	48 kbps
wifi_weak	RTT < 60 ms, > 5	1280 px	2 MB	32 kbps
slow	otherwise	800 px	1 MB	24 kbps

When a user has several connections, the server uses the worst active tier.

6. Security Architecture

Security is the heart of the project. This section describes the threat model, the key infrastructure, every encryption path, and a summary of exactly what the server can and cannot see.

6.1 Threat Model and Assumptions

- Adversary on the wire. Someone with access to the LAN segment may observe or modify traffic. Mitigated by HTTPS/WSS (Section 6.9).
- Curious or compromised server. The server operator, or an attacker who fully compromises the server, may read the database and the uploads directory. Mitigated by end-to-end encryption: the server only ever holds ciphertext and sealed key material (Section 6.10).
- Active man-in-the-middle on key distribution. An attacker who can substitute public keys could impersonate a contact. Mitigated by out-of-band fingerprint verification (Section 6.6).
- Out of scope. Endpoint compromise (malware on a user's own device), denial of service by a fully trusted insider, and traffic-analysis metadata (who talks to whom, and when) are not defended against.

6.2 Identity and Key Infrastructure

Each user owns a long-term ECDH P-256 key pair, generated in the browser with the Web Crypto API on first use. The private key is stored locally (as a JWK in localStorage) and never transmitted to the server. The public key (SPKI,

base64) is registered with the server on every authenticated page load so that other users can encrypt to it. Optionally, the private key can be backed up to the server encrypted under a PBKDF2-derived key from the user's password, enabling multi-device use without ever exposing the private key in the clear.

6.3 Message Encryption (ECIES + AES-256-GCM)

For every direct message:

- The sender generates a fresh ephemeral P-256 key pair.
- ECDH between the ephemeral private key and the recipient's long-term public key yields a shared secret, derived into an AES-256-GCM key.
- The plaintext is encrypted under a random 12-byte IV.
- The ciphertext, the ephemeral public key, and the IV are sent to the server and stored. The AES key itself is never stored or transmitted.
- The recipient reconstructs the same AES key from their private key and the ephemeral public key, and decrypts.

Because each message uses a fresh ephemeral key, compromise of one message's ephemeral key does not affect others. Broadcast messages are intentionally plaintext - they are public to the whole network and have no single recipient to encrypt to.

6.4 File Encryption Paths

There are two encrypted file paths plus a plaintext path:

- Client E2EE files (/upload/e2ee, direct messages): the browser generates a random AES-256-GCM file key, encrypts the file locally, ECIES-seals the file key for the recipient (and separately for the sender), and encrypts the file metadata (name, type, size). Only ciphertext and sealed key material reach the server.
- Server-at-rest encryption (video): video must be transcoded server-side, so the server necessarily sees the plaintext during transcoding; afterwards it encrypts the output at rest, ECIES-sealed to the recipient. This is a deliberate, documented weaker point limited to video.
- Plaintext (broadcast files and legacy small files): stored as-is, since they are public.

6.5 FastTransfer Security (Large Files)

Large direct files use the FastTransfer protocol (Section 7.4). Its security

properties:

- A fresh AES-256-GCM key is generated per transfer and **ECIES-sealed to the recipient**; it travels over the WebSocket signalling channel only.
- The file is split into shards; **each shard is independently AES-256-GCM encrypted** with its own 12-byte IV before leaving the browser.
- The file name, type, and size live in an encrypted metadata blob - the server never learns them.
- The server stores and serves only ciphertext shards, authorised so that only the registered sender may upload and only the registered recipient may download.

6.6 Fingerprint Verification (Anti-MITM)

Public-key distribution through the server is the one place an active attacker could substitute a key. To defend against this, each public key has a short, human-readable fingerprint (exposed via `/api/me/fingerprint` and computed client-side by the Fingerprint module). Two users can read their fingerprints to each other over a trusted out-of-band channel (in person, by phone) and mark the contact verified. The client records the last-seen key per contact and raises a `fingerprint:mismatch` warning if a verified contact's key ever changes - the classic signal of a man-in-the-middle or a re-keyed (possibly impersonated) account.

6.7 Voice Call and Voice Message Security

Live voice calls use WebRTC, whose media is encrypted in transit by mandatory DTLS-SRTP; the IntraComms server relays only the call signalling (`call_invite`, `call_accept`, `call_decline`, `call_ice`, `call_end`) and never carries the audio. Voice messages are recorded in the browser, then encrypted and uploaded through the same client-E2EE file path as any other direct file, so they are end-to-end encrypted at rest.

6.8 Authentication and Session Security

Passwords are hashed with PBKDF2-HMAC-SHA256 and a per-password random salt (Werkzeug). Sessions are managed by Flask-Login with a server-signed cookie. Every non-public route is protected by `@login_required`; admin routes additionally require `role == "admin"`. Deactivated accounts cannot log in, and an administrator cannot deactivate or demote their own account (preventing accidental lock-out).

6.9 Transport Security

On first start the server generates a self-signed X.509 certificate (RSA-2048, SHA-256) with localhost, 127.0.0.1, and the machine's LAN IP in the Subject Alternative Names, and serves everything over HTTPS/WSS. A secure context is also a hard requirement for the browser features the app relies on (Web Crypto, the Notification API, and the File System Access API used by FastTransfer). Because the certificate is self-signed, each device must accept it once.

6.10 What the Server Knows

Data	Stored on server	Server can read?
Direct message text	Ciphertext	No
Broadcast message text	Plaintext	Yes (public by design)
Direct file contents	Ciphertext shards/blob	No
Direct file name / type	Encrypted metadata	No
Per-file / per-transfer key	ECIES-sealed only	No
User private key	Absent, or pw-encrypted	No
User public key, presence	Plaintext	Yes (needed to function)
Password	PBKDF2 hash	No
Voice-call audio	Not stored (P2P)	No

6.11 CIA-Triad Mapping

- Confidentiality - ECIES + AES-256-GCM for messages and direct files; HTTPS in transit; PBKDF2 for passwords; private keys never leave the browser.
 - Integrity - AES-GCM authentication tags detect tampering; SQLAlchemy parameterises all queries; secure_filename and an extension allowlist guard uploads.
 - Availability - per-user rate limiting, automatic reconnection with exponential backoff and an offline send queue, background cleanup of stale data, and soft deletes that avoid destructive cascades.
-

7. Feature Implementation

7.1 Registration and Authentication

POST /register accepts a username, email, and password (and optionally a public key). The first user to register is automatically made an administrator; everyone after is a regular user. A successful registration broadcasts a user_joined event so all connected clients add the newcomer to their Contacts panel without reloading. POST /login validates credentials and the is_active flag and establishes the session; POST /logout clears it.

7.2 Messaging

The dashboard loads the most recent relevant messages once; the client filters them by active conversation purely in the DOM. Direct messages are end-to-end encrypted (Section 6.3); broadcast messages go to the broadcast room in plaintext. A live character counter enforces the per-message limit, the sidebar shows a live preview of each conversation's latest message, and a sender may soft-delete their own messages (message_deleted is pushed so all clients show "Message deleted" instantly).

7.3 File and Media Pipeline

Uploaded files pass through POST /upload (images, audio, documents), POST /upload/video/chunk (videos, in adaptive chunks), or POST /upload/e2ee (client-encrypted direct files). The server applies secure_filename, an extension allowlist, and a 50 MB cap, then:

```
upload
|
+-----+-----+
| image (Pillow)  re-encode to WebP, |
|                  strip EXIF, thumb |
+-----+-----+
| audio (FFmpeg)  re-encode to OGG   |
+-----+-----+
| video (chunked) assemble -> ffprobe |
|                  -> ffmpeg transcode |
|                  -> encrypt at rest  |
|                  -> emit media_ready |
+-----+-----+
| other           store as-is        |
+-----+-----+
```

Video is processed on a background thread; the message bubble appears immediately with a "processing" placeholder and is upgraded in place when media_ready arrives.

7.4 FastTransfer - Large-File Transfer

This subsystem was redesigned during the project (see Section 11) and is the most substantial piece of original engineering. It replaces an earlier WebRTC peer-to-peer data-channel approach that stalled and ran slowly when the two peers were on different network segments (one wired, one wireless).

Design. FastTransfer routes the file through the server as a stream of independently encrypted shards over ordinary parallel TCP connections - the server is the one host both clients can always reach. It is store-and-forward, not a live pipe: the sender uploads each finished encrypted shard to a temporary file, and the receiver downloads finished shards (receiving HTTP 425 "Too Early" and retrying while a shard is still uploading). This pattern is required because the server runs eventlet without monkey-patching, so a request that blocked waiting for another request would freeze the whole server.

Sender	Server (blind relay)	Receiver
-----	-----	-----
gen AES key; ECIES-seal to recipient		
encrypt metadata (name/type/size)		
--- transfer_offer (WS) ----->		show Accept/Decline
{ token, n_shards, sealed_key, meta }		
		[Accept]
		<----- transfer_accept (WS)
for each shard (4 at a time):		
slice -> AES-GCM encrypt		
POST /transfer/<token>/shard/<i> --->	store <i>.part	
	rename -> <i>	GET /transfer/.../<i>
	(425 until ready) <-----	(retry on 425)
	send ciphertext ----->	decrypt shard
		write to disk / buffer
--- (upload done) ----->		
	purge shards <-----	transfer_complete (WS)

Key properties and limits.

- Shards are at most 32 MB (kept under the server's request cap); up to 2048 shards are allowed, giving a 64 GB ceiling - effectively no limit for LAN use.
- Concurrency is bounded (four shards in flight) so peak memory stays bounded on the sender regardless of file size.
- The receiver chooses a delivery strategy by size: files up to 256 MB are buffered in memory and auto-downloaded; **larger files are streamed shard-by-shard straight to disk** via the browser's File System Access API, holding only about two shards in memory at once. On browsers without that API, files over 512 MB are refused with a clear message rather than risking a tab crash.

- The transfer is consent-gated (Accept/Decline) and the server purges staged shards on completion, decline, error, or a 30-minute inactivity timeout; activity on a shard refreshes the timer so long transfers are never purged mid-flight.
- End-to-end encryption is preserved throughout (Section 6.5).

7.5 Voice Messages

A user can record an audio note in the browser; it is encrypted client-side and uploaded through the E2EE file path, so it is end-to-end encrypted. Historical voice notes render as inline players that decrypt on demand.

7.6 Voice Calls

Real-time voice calls use WebRTC. The caller sends a `call_invite`; the callee may `call_accept` or `call_decline`; ICE candidates are exchanged via `call_ice`; and either party ends with `call_end`. Audio flows peer-to-peer (DTLS-SRTP encrypted) and never touches the server.

7.7 Desktop Notifications

The Notify module uses the browser Notification API to alert the user about incoming direct and broadcast messages, file offers, transfer results, voice calls, and new-user registrations - but only while the application window is not focused (so the in-app UI suffices when the user is looking). Clicking a notification focuses the window and opens the relevant conversation. A bell button in the navbar requests permission and toggles mute, persisted in `localStorage`.

7.8 Presence

Presence is tracked by a per-user connection counter (Section 5.4.2), correctly handling multiple tabs, with online/offline broadcasts and an on-connect snapshot.

7.9 Administration

The admin panel lists all users with their status and lets an administrator activate, deactivate, or change the role of any account, subject to the self-protection rules in Section 6.8.

7.10 Message Deletion

Deletion is a soft delete: the record is flagged rather than removed, preserving referential integrity and enabling potential recovery, and all clients are notified to update the UI immediately.

8. Implementation Details

8.1 Project Structure

```
IntraComms/
  server/
    server.py          app factory, all routes, all socket handlers
    models/
      database.py      shared SQLAlchemy db instance
      user.py          User model
      message.py       Message model
      file_record.py   FileRecord model
    templates/         Jinja2 templates (base + 5 pages)
    static/
      css/app.css      single stylesheet (design tokens as CSS vars)
      js/
        chat.js        main client (messaging, E2EE, UI)
        fast_transfer.js large-file sharded transfer
        voice_chat.js  WebRTC voice calls
        voice_message.js recorded voice notes
        fingerprint.js key verification
        notifications.js desktop notifications
    uploads/           encrypted/processed file storage
    intracomms.db       SQLite database
    requirements.txt
```

8.2 Technology Stack

Layer	Technology	Version	Purpose
Web framework	Flask	3.1.1	Routing, Jinja2 templates
Real-time	Flask-SocketIO	5.5.1	WebSocket events and
Async worker	eventlet	0.39.1	Async I/O for Socket.IO rooms
ORM	Flask-SQLAlchemy	3.1.1	Database access
Database	SQLite	built-in	Local persistent storage
Auth	Flask-Login	0.6.3	Sessions, login_required

Layer	Technology	Version	Purpose
Cryptography	cryptography	44.0.3	X.509, key handling
Images	Pillow	>= 10.0	WebP re-encode, EXIF, thumbnails
Video / audio	FFmpeg	external	Transcode video and audio
Scheduler	APScheduler	>= 3.10	Background cleanup
Security helpers	Werkzeug	3.1.3	Password hashing,
Config	python-dotenv	1.1.0	.env loading
Frontend	HTML5 + CSS3 + JS	-	UI, no framework
Client crypto	Web Crypto API	-	ECDH P-256, AES-256-GCM

8.3 Notable Engineering Decisions

- Cache-busting. A template helper appends each static asset's file modification time as a `?v=<mtime>` query parameter. Without this, LAN clients can keep stale cached JavaScript after a server update, and two peers running different protocol versions break file transfers. This was added after exactly that bug was observed.
- eventlet without monkey-patching. The app runs eventlet but does not call `eventlet.monkey_patch()`, because doing so on the existing, working app risks destabilising SQLAlchemy/SQLite. This constraint is the reason FastTransfer uses a non-blocking store-and-forward design rather than a blocking in-memory pipe.
- Idempotent migrations. Schema upgrades are applied as guarded ALTER TABLE statements at start-up, avoiding a migration framework for a single-file database.
- In-process state. The rate limiter, presence counters, connection-tier table, and transfer registry are in-process dictionaries - appropriate for a single-server LAN deployment, and a known scaling limit (Section 12).

9. Testing and Validation

9.1 Strategy

Testing combined automated integration tests for the highest-risk new code

(the FastTransfer server routes), an end-to-end transfer test through a running server, and a manual test matrix for user-facing behaviour.

9.2 Automated FastTransfer Route Tests

A Python integration test exercised the shard routes against the real application using the Flask test client and two logged-in users. All thirteen checks passed:

Check	Expected	Result
Sender uploads a shard	200	Pass
Sender attempts to download (not the recipient)	403	Pass
Recipient attempts to upload (not the sender)	403	Pass
Recipient downloads a shard	425	Pass
Recipient downloads an uploaded shard	200, bytes ok	Pass
Out-of-range shard index	400	Pass
Malformed transfer token	400 / 404	Pass
Unauthenticated request	redirect/401	Pass
Registry purge frees the transfer	gone	Pass
Upload/download at shard index	200, bytes ok	Pass
2047 (new 2048 cap)		

9.3 End-to-End Transfer Verification

A self-transfer was run through the live server in two configurations. A 40 MB file exercised the in-memory delivery path; a 257 MB file (nine shards) exercised the streaming-to-disk path. In both cases the reassembled file was **byte-for-byte identical** to the original (verified by length and content hash), shards were written in the correct order, and the server automatically purged the staged shards afterwards.

9.4 Manual Test Matrix

Scenario	Result
First user becomes admin; second is a regular user	Pass

Scenario	Result
Login / logout session lifecycle	Pass
Broadcast message reaches all connected clients	Pass
Direct message reaches only sender and recipient	Pass
E2EE direct message: ciphertext in DB, plaintext in	Pass
Offline queue: message sent while disconnected	Pass
Image upload re-encoded to WebP with thumbnail delivered later	Pass
Video chunk upload assembled, transcoded,	Pass
E2EE file upload stored as ciphertext, decrypted on	Pass
Large file via FastTransfer (in-memory and streaming	Pass
Voice message recorded, encrypted, played back (paths)	Pass
Voice call connects and carries audio	Pass
Desktop notification fires only when window unfocused	Pass
Fingerprint mismatch warning on a changed key	Pass
Admin deactivate prevents login	Pass
Rate limit triggers after the per-minute threshold	Pass
Message delete shows "Message deleted" for both	Pass
Self-signed TLS accepted after one-time exception	Pass
Mobile layout: rail/panel toggle on small screens	Pass

9.5 Security Validation

- Inspecting the database confirmed that direct-message content and E2EE file blobs are ciphertext, and that no private keys or file keys are stored in the clear.
- Tampering with a stored ciphertext caused AES-GCM decryption to fail (the client shows "Unable to decrypt"), confirming integrity protection.
- All database access is through SQLAlchemy's parameterised ORM, and uploads are constrained by secure_filename plus an extension allowlist.

10. Results and Discussion

IntraComms meets every objective in Section 1.3. It delivers real-time messaging, default end-to-end encryption for direct communication, a full media pipeline, unlimited-size encrypted file transfer, voice messages and calls, fingerprint-based identity verification, and desktop notifications - all on a single self-hosted process with no internet dependency.

The most important qualitative result is that the server is genuinely blind to private content: the security validation confirmed that direct messages, direct file contents, file names, and keys are never available to the server in plaintext. The most important quantitative result is that the redesigned FastTransfer subsystem transfers files of arbitrary size correctly (byte-exact in testing) and reliably across mixed wired/wireless segments where the previous WebRTC approach failed.

A fair caveat on throughput: the figures observed during development came from a single-machine self-loopback against the single-threaded development server, which is not representative of a true two-device LAN transfer; real-world throughput on separate machines is expected to be substantially higher and remains to be benchmarked.

11. Challenges and Solutions

- WebRTC large-file transfer stalled across segments. With no STUN/TURN, WebRTC depends on ICE finding a direct host-candidate path, which degraded or failed when peers were on different subnets, and SCTP congestion control added overhead on the low-latency LAN. Solution: abandon peer-to-peer for files and relay encrypted shards through the server over plain TCP (FastTransfer), which both clients can always reach.
- A 256 KB message froze the receiver. An intermediate design sent oversized data-channel messages that exceeded the browser's SCTP message-size limit and aborted the channel. Solution (superseded by FastTransfer): cap message size and, ultimately, move off data channels entirely.
- eventlet would freeze under a blocking pipe. A naive server-relay design using a blocking in-memory queue would have frozen the whole server because eventlet is not monkey-patched. Solution: a non-blocking store-and-forward design (HTTP 425 + retry) that mirrors the existing, proven upload/download routes.
- Receiver memory limited file size. Buffering the whole decrypted file in RAM

before saving capped transfers at a few hundred megabytes. Solution: stream shards straight to disk via the File System Access API, reducing peak memory to about two shards and lifting the practical ceiling to tens of gigabytes.

- Stale cached clients caused version mismatches. Two peers on different cached versions of the client broke transfers. Solution: automatic cache-busting of all static assets by modification time.

12. Limitations

Limitation	Detail
Single server	All shared state is in-process; no clustering
SQLite	Fine for a LAN node; PostgreSQL needed for high concurrency
Broadcast is plaintext	Group E2EE would require key distribution not covered
Per-message ECIES, no ratchet	Not a Signal-style double-ratchet; no post-compromise security
Video encrypted only at rest	Server sees video plaintext during transcoding recovery
Streaming receive needs Chromium	The File System Access API is unavailable in some browsers
Notifications need an open tab	No background push without a service worker / internet browsers
No message search or pagination	History is loaded in a bounded window and not searchable

13. Future Work

- Double-ratchet messaging (Signal Protocol) for post-compromise security.
- Group end-to-end encryption via Sender Keys or MLS for encrypted broadcasts.
- PostgreSQL backend (drop-in via DATABASE_URL) and externalised shared state for multi-server deployment.
- Real-device throughput benchmarking of FastTransfer and tuning of shard size and concurrency.
- Read receipts and typing indicators, using the existing Socket.IO plumbing.
- Message search, client-side for E2EE content and server-side for broadcasts.
- A non-Chromium streaming fallback for large-file receive (for example via a service-worker download stream).

14. Conclusion

IntraComms demonstrates that a small, single-developer project can deliver a genuinely secure, real-time communication platform that runs entirely within a LAN. By placing all cryptography in the browser and keeping the server blind to plaintext, the system provides default end-to-end encryption for direct messages and files while remaining a single, easily deployed Python process. The project also shows the value of iterative engineering: the large-file transfer subsystem was rebuilt from a fragile WebRTC peer-to-peer design into a robust, server-relayed, end-to-end-encrypted protocol that streams files of effectively unlimited size to disk, after the original design's failure modes were diagnosed and understood.

The result satisfies all stated objectives and is structured to remain readable and maintainable - a backend concentrated in one orchestrating module, a clean model package, and a set of focused client modules - making it both a working tool and a clear teaching artefact for web security, real-time communication, and media processing.

15. References

- T. Dierks and E. Rescorla, The Transport Layer Security (TLS) Protocol, RFC 8446, IETF, 2018.
 - M. Thomson and C. Jennings, WebRTC: Real-Time Communication in Browsers, W3C / IETF RTCWEB, 2021.
 - R. Jesup, S. Loreto, and M. Tuexen, WebRTC Data Channels, RFC 8831, IETF, 2021.
 - National Institute of Standards and Technology, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM)*, NIST SP 800-38D, 2007.
 - Certicom Research, SEC 1: Elliptic Curve Cryptography (ECIES), v2.0, 2009.
 - B. Kaliski, PKCS #5: Password-Based Cryptography Specification (PBKDF2), RFC 2898, IETF, 2000.
 - Pallets Projects, Flask Documentation and Werkzeug Documentation, 2024.
 - M. Grinberg, Flask-SocketIO Documentation, 2024.
 - Mozilla Developer Network, Web Crypto API and File System Access API, 2024.
-

Appendix A: HTTP API Reference

Method	Path	Auth	Description
GET	/	No	Landing page (redirects if
GET/POST	/register	No	User registration
GET/POST	/login	No	User login
POST	/logout	Yes	Logout
GET	/dashboard	Yes	Main chat UI
POST	/messages/<id>/delete	Yes	Soft-delete a message
POST	/upload	Yes	(send only) Upload a file, image, audio, or
POST	/upload/video/chunk	Yes	Upload one video chunk
POST	/upload/e2ee	Yes	Upload a client-encrypted
GET	/uploads/<id>	Yes	Serve a plaintext file
GET	/uploads/<id>/thumb	Yes	Serve a thumbnail
GET	/uploads/<id>/blob	Yes	Serve an E2EE encrypted
GET	/uploads/<id>/meta	Yes	Return E2EE key material
POST		Yes	Upload one encrypted
GET	/transfer/<token>/shard/<n>	Yes	shard (sender) one shard
POST	/api/me/pubkey	Yes	Register the user's public
GET	/api/me/fingerprint	Yes	key Get the user's key
GET/POST	/api/me/key-backup	Yes	Fetch / store encrypted
GET	/api/users/<id>/pubkey	Yes	private key Get other user's public
GET	/admin	Admin	User-management panel
POST	/admin/users/<id>/activate	Admin	Activate a user
POST			
	/admin/users/<id>/deactivat		

e

		Admin	Deactivate a user
POST	/admin/users/<id>/role	Admin	Change a user's role
GET	/ping	No	Latency probe

Appendix B: Socket.IO Event Reference

Client to server

Event	Purpose
message	Send a message (typed envelope)
client_tier	Report connection quality
transfer_offer	Offer a FastTransfer (register + relay)
transfer_accept	Accept an incoming transfer
transfer_decline	Decline an incoming transfer
transfer_complete	Signal a transfer finished (purge)
transfer_error	Signal a transfer error (purge)
call_invite	Start a voice call
call_accept	Accept a voice call
call_decline	Decline a voice call
call_ice	Exchange a WebRTC ICE candidate
call_end	End a voice call

Server to client (typed envelope inside the message event)

Type	Purpose
new_message	A new message arrived
message_deleted	A message was deleted
media_ready	Video transcode completed
media_failed	Video transcode failed
user_online	A user came online
user_offline	A user went offline
online_users	Presence snapshot on connect
user_joined	A new user registered
client_config	Server-assigned upload configuration
transfer_offer	Incoming file offer (sealed key + meta)
transfer_accept	Recipient accepted (sender UI)
transfer_decline	Recipient declined
transfer_complete	Recipient finished downloading
call_invite/call_accept/call_decline/call_ice/call_end	Voice-call signalling
error	Error response

(A set of legacy p2p_ signalling events from the earlier WebRTC file-transfer design remains in the server for reference but is no longer used by the client.)*

Appendix C: Database Schema Summary

Table	Key columns
users	id, username, email, password_hash, role, public_key,
messages	id, sender_id, receiver_id, content, ephemeral_pub, iv,
file_records	sender_id, copy_id, message_id, receiver_id, is_original, filtered, stored_filename, file_size, mime_type, visibility, status,

Appendix D: Setup and Deployment Guide

```
# 1. Create and activate a virtual environment
python -m venv .venv
.venv\Scripts\Activate.ps1          # Windows PowerShell

# 2. Install dependencies
pip install -r requirements.txt

# 3. Initialise the database (first run only)
python -m flask --app server/server.py init-db

# 4. Run the server (auto-generates a TLS certificate on first start)
python server/server.py
```

The server listens on 0.0.0.0:5000 and is reachable from any device on the LAN at https://<host-ip>:5000. Each device accepts the self-signed certificate once.

Environment variables

Variable	Default	Purpose
SECRET_KEY	dev-secret-change-me	Flask session signing key
DATABASE_URL	sqlite:///intracomms.db	SQLAlchemy connection string
FLASK_DEBUG	0	Enable debug mode and reloader
PORT	5000	Listening port

CLI commands

Command	Description
flask --app server/server.py init-db	Create all database tables
flask --app server/server.py clear-db	Drop and recreate all tables
flask --app server/server.py promote-admin <username>	Grant a user the admin role

Report prepared June 2026. Project repository: D:\IntraComms